# Guia de Tarefas e Execução Técnica - Time B (Projeto MAFE)O que o Grupo Gamma (POO & Back-End em Python Flask) DEVE FAZER

## 1.1. Mapeamento e Transformação do Banco em Classes/Objetos (SQLAlchemy)

O Grupo Gamma é responsável por pegar o banco MySQL (`mafe_db`) e transformar cada tabela em uma classe (Objeto Python) usando SQLAlchemy, aplicando Programação Orientada a Objetos:

- **Classe** `Categoria`**:** Mapeia `categorias_galeria`. Atributos: `id`, `nome`. Criar métodos POO: `cadastrar()`, `atualizar()`, `deletar()`, `listar_todas()`.
- **Classe** `Midia`**:** Mapeia `midias`. Atributos: `id`, `categoria_id`, `titulo`, `descricao`, `url_midia`, `tipo_midia`, `ano`, `visualizacoes`, `ativo`, `criado_em`. Criar métodos POO: `cadastrar()`, `atualizar()`, `alternar_status()` (soft delete), `listar_todas_admin()`.
- **Classe** `Apoiador`**:** Mapeia `apoiadores`. Atributos: `id`, `nome_empresa`, `descricao`, `logo_url`, `site_url`, `maps_url`, `ativo`. Criar métodos POO: `cadastrar()`, `atualizar()`, `alternar_status()`, `listar_ativos()`, `listar_todos_admin()`.
- **Classe** `EstatisticaGrupo`**:** Mapeia `estatisticas_grupo`. Atributos: `id`, `quantidade_participantes`, `quantidade_torneios`, `ano_fundacao`. Criar métodos POO: `obter_estatisticas()`, `atualizar_metricas()`.

## 1.2. Segurança e Hash de Senha (Werkzeug)

O Grupo Gamma deve configurar a proteção da senha do admin único (sem criar tabelas desnecessárias no banco):

- Criar o arquivo `config.py` guardando `ADMIN_USER = "coordenacao"` e `ADMIN_PASSWORD_HASH`.
- Gerar o hash usando `generate_password_hash` da biblioteca `werkzeug.security`.
- Criar a rota `POST /api/login` que recebe usuário e senha digitados e valida via `check_password_hash`.

## 1.3. Criação das Rotas Exclusivas do Painel Administrativo

*Nota: As rotas públicas de consulta do site público serão feitas por outro desenvolvedor. O Grupo Gamma é responsável estritamente pelos endpoints do Painel Admin.*

- **Login:**`POST /api/login`
- **Galeria/Mídias:**`GET /api/admin/midias`, `POST /api/admin/midias`, `PUT /api/admin/midias/<id>`, `PATCH /api/admin/midias/<id>/status`.

- **Categorias:**`GET /api/admin/categorias`, `POST /api/admin/categorias`, `PUT /api/admin/categorias/<id>`, `DELETE /api/admin/categorias/<id>`.
- **Apoiadores:**`GET /api/admin/apoiadores`, `POST /api/admin/apoiadores`, `PUT /api/admin/apoiadores/<id>`, `PATCH /api/admin/apoiadores/<id>/status`.
- **Estatísticas:**`GET /api/admin/estatisticas`, `PUT /api/admin/estatisticas`.

### 1.4. Requisitos Técnicos de Integração e Resposta

- **Configuração de CORS (`flask-cors`):** Instalar e configurar a extensão `flask-cors` no aplicativo Flask para liberar requisições originadas do front-end (React) em ambiente de desenvolvimento local, evitando bloqueios do navegador.
- **Padronização de Respostas JSON e Status HTTP:** Garantir que todas as respostas dos endpoints do painel retornem uma estrutura JSON padronizada com indicação clara do status da operação, tais como `{"sucesso": true, "dados": [...], "mensagem": "..."}` e os devidos códigos HTTP (`200` para sucesso, `400` para dados inválidos, `401` para não autorizado, `500` para erro interno).
- **Validação de Entradas (Inputs):** Verificar se os campos obrigatórios enviados pelo front-end (ex: `titulo`, `url_midia`, `nome_empresa`) estão preenchidos antes de tentar persisti-los no banco de dados, retornando erros amigáveis em vez de exceções de banco não tratadas.
- **Tratamento de Chaves Estrangeiras na Exclusão:** No método de exclusão de categorias, verificar ou tratar a restrição de chave estrangeira (`ON DELETE RESTRICT`) do MySQL para impedir a exclusão de categorias com mídias associadas, devolvendo uma mensagem explicativa para o administrador.

### 1.4. Requisitos Técnicos de Integração e Respostas da API

- Configuração de CORS (flask-cors): Habilitar o CORS na aplicação Flask para permitir requisições cross-origin vindas do endereço local do React (ex: localhost:5173).
- Padronização de Respostas JSON e Status HTTP: Garantir que todas as respostas retornem JSON com estrutura padrão (ex: {"sucesso": true, "dados": [...], "mensagem": "..."}) e os devidos códigos de status HTTP (200 para sucesso, 400 para dados inválidos, 401 para não autorizado, 500 para erro interno).
- Validação de Inputs Recebidos: Tratar e validar se campos obrigatórios (ex: titulo, url_midia, nome) não estão vazios ou nulos antes de persistir no banco de dados.
- Tratamento de Exclusão com Chave Estrangeira (FK): Tratar a exceção do MySQL ao tentar deletar categorias que possuam mídias vinculadas (ON DELETE RESTRICT), retornando uma mensagem explicativa no JSON de resposta.

## 2. O que o Desenvolvedor Front-End (Grupo Alfa / Painel Admin em React) DEVE FAZER

### 2.1. Tela de Login e Fluxo de Autenticação

- Criar a interface de Login (campo de usuário, senha e botão 'Entrar').
- Enviar a requisição para `POST /api/login`.
- Se o login for bem-sucedido, salvar o estado de autenticação (ex: no `localStorage` ou contexto do React) e redirecionar para o Dashboard.
- Proteger as rotas do Painel Admin no React Router para proibir acesso não autenticado.

### 2.2. Organização e Estrutura das Abas do Painel Admin

O painel administrativo deve ser organizado em abas/seções claras para a coordenação gerenciar o site:

- **Aba Galeria de Mídias:**
  - Tabela/Grade exibindo todas as fotos e vídeos cadastrados (com indicação se está visível ou oculta).
  - Botão "Adicionar Nova Mídia" (abre formulário com título, descrição, URL, ano, categoria e tipo).
  - Botão de Ação "Editar" e botão "Ocultar/Exibir" (dispara o Soft Delete no Flask).
- **Aba Categorias:**
  - Lista das categorias existentes (Treinos, Liga Sorocabana, etc.).
  - Formulário simples para cadastrar nova categoria ou editar nome existente.
- **Aba Apoiadores:**
  - Lista de parceiros com logo, nome e status (`ativo`).
  - Formulário para cadastrar/editar parceiros (campos: nome, descrição, logo, link do site e link do Google Maps).
  - Botão para desativar/reativar o apoiador no site.
- **Aba Estatísticas do Grupo:**
  - Formulário simples para atualizar o número total de participantes e o número de torneios.

### 2.3. Conexão do React com o Flask (Services)

- Criar funções utilitárias (usando `axios` ou `fetch`) mapeando as requisições para os endpoints `/api/admin/*` criados pelo Grupo Gamma.
- Implementar feedbacks visuais de carregamento (loading) e mensagens de sucesso/erro para o usuário ao salvar ou alterar qualquer dado